Universidade Estadual de Santa Cruz – UESC

Relatórios de Implementações de Métodos da Disciplina Análise Numérica

Relatório de implementações realizadas por Ariel Pina Ribeiro

Disciplina Análise Numérica.

Curso Ciência da Computação

Semestre 2025.2

Professor Gesil Sampaio Amarante II

Ilhéus – BA
2025

ÍNDICE

ÍNDICE **2**
Linguagem(ns) Escolhida(s) e justificativas **6**
Método da Bissecção **7**
Estratégia de Implementação: 7
Estrutura dos Arquivos de Entrada/Saída 7
Dificuldades enfrentadas 8
Método da Posição Falsa **9**
Estratégia de Implementação: 9
Estrutura dos Arquivos de Entrada/Saída 9
Dificuldades enfrentadas 10
Método de Newton - Raphson **11**
Estratégia de Implementação: 11
Estrutura dos Arquivos de Entrada/Saída 11
Dificuldades enfrentadas 12
Método da Secante **13**
Estratégia de Implementação: 13
Estrutura dos Arquivos de Entrada/Saída 13
Dificuldades enfrentadas 14
Problemas Teste (Propostos pelo professor) **15**
Problema 3.3: **15**
Entrada 1: 15
Entrada 2: 15
Saída por Bissecção(Entrada 1): 15
Saída por Posição Falsa(Entrada 1): 16
Saída por Newton-Raphson(Entrada 1): 16
Saída por Secante(Entrada 1): 16
Saída por Bissecção(Entrada 2): 16
Saída por Posição Falsa(Entrada 2): 16
Saída por Newton-Raphson(Entrada 2): 16
Saída por Secante(Entrada 2): 17
Comparação de erros(Entrada 1): 17
Comparação de iterações(Entrada 1): 18
Comparação de tempo(Entrada 1): 18
Comparação de erros(Entrada 2): 19
Comparação de iterações(Entrada 2): 19
Comparação de tempo(Entrada 2): 20
Problema 3.6: **21**
Entrada 1: 21
Saída por Bissecção(Entrada 1): 21

Saída por Posição Falsa(Entrada 1):.....21
Saída por Newton-Raphson(Entrada 1):.....21
Saída por Secante(Entrada 1):.....22
Comparação de erros(Entrada 1):.....22
Comparação de iterações(Entrada 1):.....23
Comparação de tempo(Entrada 1):.....23
Problema 3.8:.....24
Entrada 1:.....24
Entrada 2:.....24
Saída por Bissecção(Entrada 1):.....24
Saída por Posição Falsa(Entrada 1):.....24
Saída por Newton-Raphson(Entrada 1):.....25
Saída por Secante(Entrada 1):.....25
Saída por Bissecção(Entrada 2):.....25
Saída por Posição Falsa(Entrada 2):.....25
Saída por Newton-Raphson(Entrada 2):.....25
Saída por Secante(Entrada 2):.....25
Comparação de erros(Entrada 1):.....26
Comparação de iterações(Entrada 1):.....26
Comparação de tempo(Entrada 1):.....27
Comparação de erros(Entrada 2):.....27
Comparação de iterações(Entrada 2):.....28
Comparação de tempo(Entrada 2):.....28
Método da Eliminação de Gauss.....29
Estratégia de Implementação:.....29
Estrutura dos Arquivos de Entrada/Saída.....29
Dificuldades enfrentadas.....30
Método da Fatoração LU.....31
Estratégia de Implementação:.....31
Estrutura dos Arquivos de Entrada/Saída.....32
Dificuldades enfrentadas.....32
Problemas Teste (Propostos pelo professor).....34
Problema 4.1:.....34
Entrada 1:.....34
Saída por Eliminação de Gauss(Entrada 1):.....34
Saída por Fatoração LU(Entrada 1):.....35
Problema 4.3:.....36
Entrada 1:.....36
Saída por Eliminação de Gauss(Entrada 1):.....36
Saída por Fatoração LU(Entrada 1):.....36
Problema 4.6:.....37
Entrada 1:.....37

Saída por Eliminação de Gauss(Entrada 1):..... 37
Saída por Fatoração LU(Entrada 1):..... 37
Método de Jacobi..... 38
Estratégia de Implementação:..... 38
Estrutura dos Arquivos de Entrada/Saída..... 39
Dificuldades enfrentadas..... 39
Método de Gauss - Seidel..... 40
Estratégia de Implementação:..... 40
Estrutura dos Arquivos de Entrada/Saída..... 41
Dificuldades enfrentadas..... 41
Condição da Matriz..... 42
Estratégia de Implementação:..... 42
Estrutura dos Arquivos de Entrada/Saída..... 42
Dificuldades enfrentadas..... 43
Problemas Teste (Propostos pelo professor)..... 44
Problema 5.1:..... 44
Entrada 1:..... 44
Condição da Matriz(Entrada 1):..... 44
Saída por Jacobi(Entrada 1):..... 45
Saída por Gauss-Seidel(Entrada 1):..... 45
Problema 5.2:..... 46
Entrada 1:..... 46
Condição da Matriz(Entrada 1):..... 46
Saída por Jacobi(Entrada 1):..... 47
Saída por Gauss-Seidel(Entrada 1):..... 47
Problema 5.5:..... 48
Entrada 1:..... 49
Condição da Matriz(Entrada 1):..... 49
Saída por Jacobi(Entrada 1):..... 49
Saída por Gauss-Seidel(Entrada 1):..... 49
Considerações Finais..... 51

Linguagem(ns) Escolhida(s) e justificativas

A linguagem escolhida para as implementações deste relatório foi a linguagem Python. A opção por essa linguagem foi por conta de sua sintaxe simples, que facilita a transformação de fórmulas matemáticas em código e o entendimento dos algoritmos. Além disso, a linguagem possui uma ampla gama de bibliotecas integradas, que facilitam as implementações dos algoritmos.

Entre essas bibliotecas, podemos utilizar NumPy para operações com vetores e matrizes, SciPy para funções numéricas prontas, SymPy quando precisarmos de manipulação simbólica e Matplotlib para gráficos. Algumas dessas bibliotecas podem precisar ser instaladas no computador ou ambiente virtual Python que estiver rodando o código.

Outro ponto importante é que Python facilita a leitura e escrita de arquivos de entrada e saída em formatos simples (texto, CSV) e a geração automática de relatórios e gráficos.

Além de ser a linguagem recomendada pelo professor, as características citadas acima tornam a linguagem adequada para implementar, testar e comparar os métodos estudados, permitindo também, se necessário, otimizações futuras sem alterar a lógica matemática das soluções.

Método da Bissecção

Estratégia de Implementação:

Para implementar o método da bissecção usei o SymPy para transformar a expressão da função em uma função numérica que pudesse ser avaliada a cada iteração. Isso resolveu a dificuldade de converter a função digitada no arquivo para algo que o código pudesse calcular de forma simples.

Implementei o critério de parada usando dois fatores: o tamanho do intervalo

$$\frac{|b-a|}{2} < tolerância$$

e o valor da função no ponto médio

$$|F(c)| < tolerância$$

além de um número máximo de iterações para evitar loops infinitos. Para medir desempenho, registrei o tempo de execução com time.time().

Também tratei erros comuns, como arquivo inexistente, problemas de conversão de a e b para número e expressões inválidas no arquivo. Quando ocorre um erro, o programa mostra uma mensagem clara no terminal para facilitar o ajuste pelo usuário.

Também padronizei o nome do arquivo de saída como saida.txt, para manter o relatório organizado e facilitar a automação de testes.

Estrutura dos Arquivos de Entrada/Saída

O arquivo de entrada (**../entrada.txt**) foi definido para ficar em um nível acima do código, simulando um cenário real onde o programa lê dados de uma pasta de trabalho separada. O formato é simples e direto:

1ª linha → expressão da função (ex.: x**3 - x - 2)

2ª linha → valor de a (limite inferior)

3ª linha → valor de b (limite superior)

4ª linha (opcional) → tolerância (padrão 1e-6)

Dificuldades enfrentadas

Durante a implementação do método da bissecção, a principal dificuldade foi garantir que o intervalo inicial realmente contivesse uma raiz, já que sem isso o método não converge. Também precisei lidar com entradas inválidas no arquivo, como funções com erros de sintaxe ou valores que não podiam ser convertidos para número. Por fim, notei que tolerâncias muito pequenas aumentavam bastante o número de iterações e o tempo de execução, exigindo atenção para escolher um valor adequado. Apesar desses pontos, o algoritmo funcionou bem e resolveu corretamente os problemas propostos.

Método da Posição Falsa

Estratégia de Implementação:

Para implementar o método da posição falsa usei SymPy para converter a expressão do entrada.txt em uma função numérica que pudesse ser avaliada facilmente no loop iterativo. Antes de entrar no laço eu avalio $f(a)$ e $f(b)$ e trato casos especiais: se $f(a)$ ou $f(b)$ for exatamente zero retorno a raiz; se $f(a) \ * \ f(b) \ > \ 0$ lanço um erro informando que o intervalo não contém mudança de sinal.

No laço principal calculo o ponto c pela fórmula da posição falsa

$$c \ = \ \frac{a*f(b) - b*f(a)}{f(b) - f(a)}$$

e, no caso raro de divisão por zero

$$f(b) \ - \ f(a) \ == \ 0$$

uso o ponto médio como fallback. Como critério de parada usei o valor absoluto da função em c

$$|F(c)| \ < \ tolerância$$

e um limite de iterações (max_iter) para evitar loops infinitos. Também medi o tempo de execução com time.time() e gravei os resultados em saida.txt.

Para tratamento de erros, adicionei verificações ao ler o arquivo (arquivo ausente, conversão de a/b inválida, etc) e capturo exceções ao avaliar $\frac{f(a)}{f(b)}$ para apresentar mensagens claras ao usuário.

Estrutura dos Arquivos de Entrada/Saída

O arquivo de entrada fica em **../entrada.txt** (nível acima do script) e tem formato simples:

1ª linha → expressão da função (ex.: x**3 - x - 2)
2ª linha → valor de a (limite inferior)
3ª linha → valor de b (limite superior)
4ª linha (opcional) → tolerância (padrão 1e-6)

O arquivo de saída fica em **saida.txt** que é gerado no diretório de execução contendo, em quatro linhas, os respectivos valores: raiz aproximada, erro final, número de iterações e tempo de execução em segundos.
Escolhi o nome fixo saida.txt por tornar os testes repetidos e a extração de resultados (para tabelas ou plots) mais simples e reprodutível, assim, scripts e automáticos podem sempre buscar o mesmo caminho sem ambiguidade.

Dificuldades enfrentadas

A principal dificuldade foi garantir que o intervalo inicial de fato contivesse uma raiz (ou seja, f(a) e f(b) com sinais opostos), porque sem isso o método não pode ser aplicado. Outra limitação prática foi lidar com situações em que f(b) - f(a) fica muito próximo de zero, nisso precisei tratar a divisão por zero dando um fallback para o ponto médio. No fim, notei que, em alguns casos, a posição falsa pode convergir lentamente quando a raiz está muito próxima a um dos extremos (o método tende a “prender” um dos pontos), e tolerâncias muito pequenas aumentam bastante o número de iterações e o tempo de execução. Apesar desses pontos, com as validações e o limite de iterações o método se mostrou estável para os problemas propostos.

Método de Newton - Raphson

Estratégia de Implementação:

Para o método de Newton–Raphson usei SymPy para transformar a expressão lida em entrada.txt em uma expressão simbólica e gerar automaticamente a sua derivada e as funções numéricas. Como chute inicial adotei o ponto médio do intervalo $x_0 = \frac{a+b}{2}$, que é uma escolha simples e funciona bem para os exemplos do trabalho.

Os critérios de parada são dois: a diferença entre iterações sucessivas

$$\left|x_{k+1} - x_k\right| < tolerância$$

e, quando possível, o valor da função no novo ponto

$$\left|F(x_{k+1})\right| < tolerância$$

além disso há um max_iter para evitar loops infinitos. Para medir desempenho registre o tempo com time.time() e gravei os resultados no arquivo de saída.

No tratamento de erros adicionei verificações ao ler o arquivo (arquivo inexistente, conversão de a/b inválida, etc) e verificações ao avaliar F e F' no chute inicial. Se a derivada for zero em algum passo o programa lança um erro claro (pois o método não pode continuar nesse ponto); exceções de avaliação são capturadas e traduzidas em mensagens legíveis para o usuário.

Estrutura dos Arquivos de Entrada/Saída

Entrada (**../entrada.txt**) — formato simples e direto para facilitar testes e automação:

1ª linha → expressão da função (ex.: x**3 - x - 2)
2ª linha → valor de a (limite inferior)
3ª linha → valor de b (limite superior)
4ª linha (opcional) → tolerância (padrão 1e-6)

Saída (**saida.txt**) — arquivo gerado no diretório de execução contendo, em quatro linhas, respectivamente: raiz aproximada, erro final, número de iterações e tempo de execução em segundos.

Dificuldades enfrentadas

A maior dificuldade prática foi a sensibilidade ao chute inicial: Newton–Raphson pode divergir ou convergir para outra raiz se o ponto inicial não for adequado, então escolher $x_0 = \frac{a+b}{2}$ é uma solução simples, mas nem sempre ideal.

Também encontrei problemas com $f' = 0$ em algum passo, tive que detectar esse caso e interromper com uma mensagem clara em vez de deixar ocorrer divisão por zero. Por fim, houve desafios com entradas mal-formadas (erros de sintaxe na expressão) e o fato de tolerâncias muito pequenas aumentarem bastante o número de iterações e o tempo de execução. Apesar disso, com as validações implementadas o método funcionou de forma estável para os exemplos propostos.

Método da Secante

Estratégia de Implementação:

Para o método da secante usei SymPy para converter a expressão do entrada.txt numa função numérica que pudesse ser avaliada dentro do laço. O algoritmo recebe dois chutes iniciais x0 e x1 e, a cada iteração, calcula o novo ponto pela fórmula da secante

$$x_{n+1} = x_n - f(x_n) * \frac{(x_n - x_{n-1})}{f(x_n) - f(x_{n-1})}$$

Antes de começar faço verificações iniciais: avalio $f(x_0)$ e $f(x_1)$ e retornos imediatos caso algum seja zero; se a diferença $f(x_1)$- $f(x_0)$ for zero durante a iteração lanço um erro (evitando divisão por zero). Como critério de parada usei a diferença entre iterações sucessivas

$$\left|x_{n+1} - x_n\right| < tolerância$$

e um max_iter para evitar loops infinitos. Para robustez capturei exceções na leitura e na avaliação de F (mensagens claras ao usuário) e medi o tempo de execução com time.time().

Estrutura dos Arquivos de Entrada/Saída

O arquivo de entrada é **../entrada.txt** (nível acima do script) e tem formato direto para facilitar testes automáticos:

1ª linha → expressão da função (ex.: x**3 - x - 2)
2ª linha → x0 (primeiro chute)
3ª linha → x1 (segundo chute)
4ª linha (opcional) → tolerância (padrão 1e-6)

Saída (**saida.txt**) — arquivo gerado no diretório de execução contendo, em quatro linhas, respectivamente: raiz aproximada, erro final, número de iterações e tempo de execução em segundos.

Dificuldades enfrentadas

A maior dificuldade foi escolher chutes iniciais adequados: secante depende fortemente de x0 e x1, e chutes ruins podem levar a divergência ou a convergência para outra raiz. Outro problema prático foi o caso

$$f(x_1) - f(x_0) \approx 0$$

que causa instabilidade (tratado no código com exceção ou mensagem ao usuário). Também surgiram situações com entradas mal-formadas (erros de sintaxe na expressão) e o fato de tolerâncias muito pequenas aumentarem muito o número de iterações e o tempo de execução. Apesar dessas limitações, com as validações implementadas o método se comportou de forma confiável para os exemplos testados.

Problemas Teste (Propostos pelo professor)

Problema 3.3:

3.3 - *Um amplificador eletrônico com acoplamento R - C com três estágios em cascata tem uma resposta a um degrau unitário de tensão dada pela expressão:*

$$g(T) = 1 - (1 + T + \frac{T^2}{2})e^{-T} \ ,$$

onde $T = \frac{t}{RC}$ *é uma unidade de tempo normalizada. O tempo de subida de um amplificador é definido como o tempo necessário para sua resposta ir de* 10% *a* 90% *de seu valor final. No caso, como* $g(\infty) = 1$ *é necessário calcular os valores de* T *para os quais*

$$g = 0.1 \quad e \quad g = 0.9$$

ou seja resolver as equações:

$$\begin{aligned} 0.1 &= 1 - (1 + T + \frac{T^2}{2})e^{-T} \ . \\ 0.9 &= 1 - (1 + T + \frac{T^2}{2})e^{-T} \ . \end{aligned}$$

Chamando de $T_{0.1}$ *o valor obtido de* T *na* 1^a *equação e* $T_{0.9}$ *o valor obtido de* T *na* 2^a *equação, calcular o tempo de subida.*

Entrada 1:

```
1 - (1 + x + x**2 / 2) * E**(-x) - 0.1
0.1
2.0
1e-6
```

Entrada 2:

```
1 - (1 + x + x**2 / 2) * E**(-x) - 0.9
3
6
1e-6
```

Saída por Bissecção(Entrada 1):

```
1.1020690917968752
0.00011596679687497 78
13
0.000051
```

Saída por Posição Falsa(Entrada 1):

```
1.1020642613381413
2.1522458026890234e-07
9
0.000024
```

Saída por Newton-Raphson(Entrada 1):

```
1.1020659163716693
0.0012020994889252457
2
0.000010
```

Saída por Secante(Entrada 1):

```
1.1020653282492539
9.80453340737597e-09
7
0.000015
```

Saída por Bissecção(Entrada 2):

```
5.32232666015625
0.00018310546875
13
0.000041
```

Saída por Posição Falsa(Entrada 2):

```
5.3223333484897015
8.995288252600098e-07
16
0.000038
```

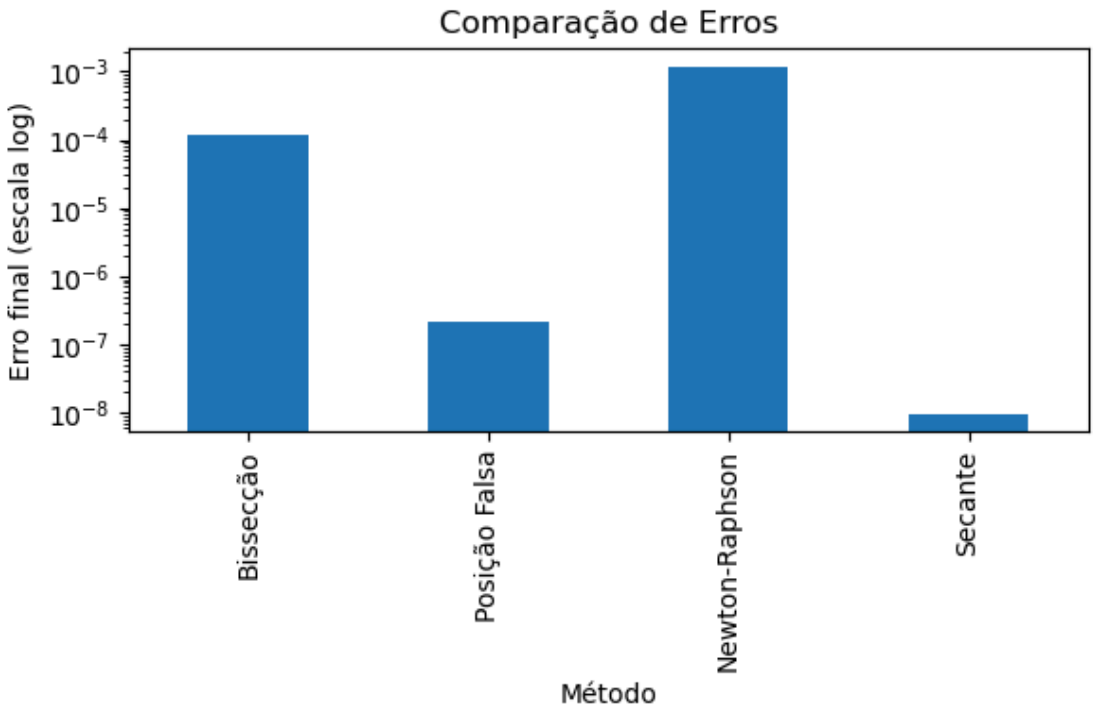
Saída por Newton-Raphson(Entrada 2):

```
5.322320337682237
2.206612169430855e-05
4
0.000015
```

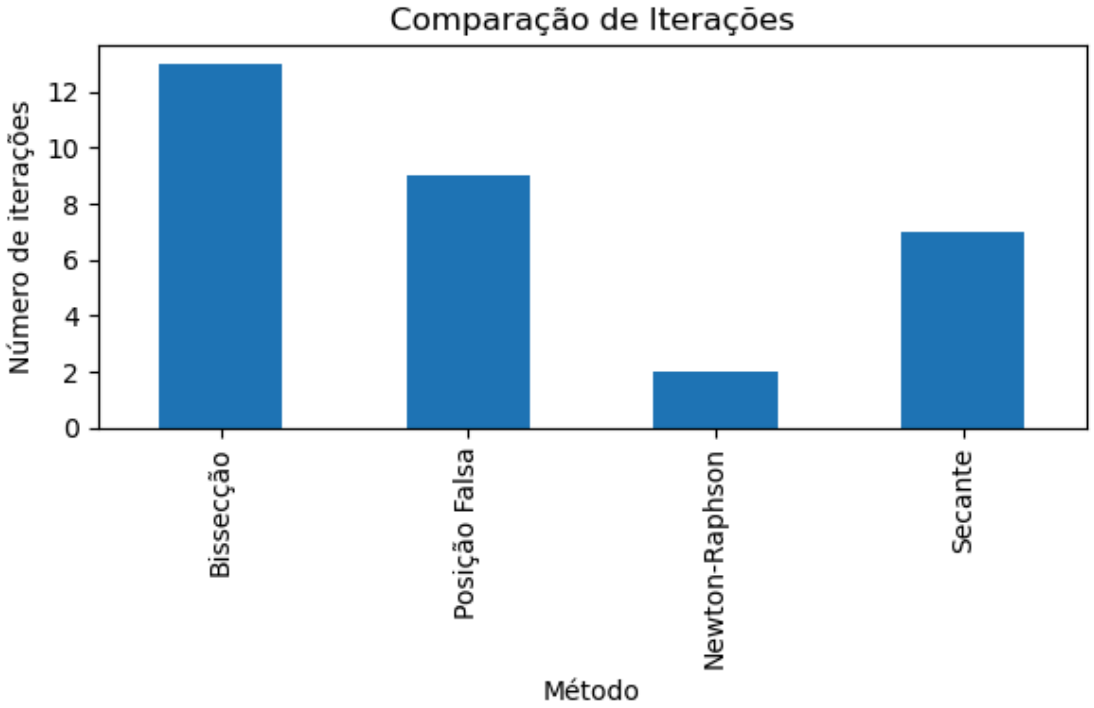
Saída por Secante(Entrada 2):

```
5.322320337917059
9.295353322613664e-07
6
0.000015
```

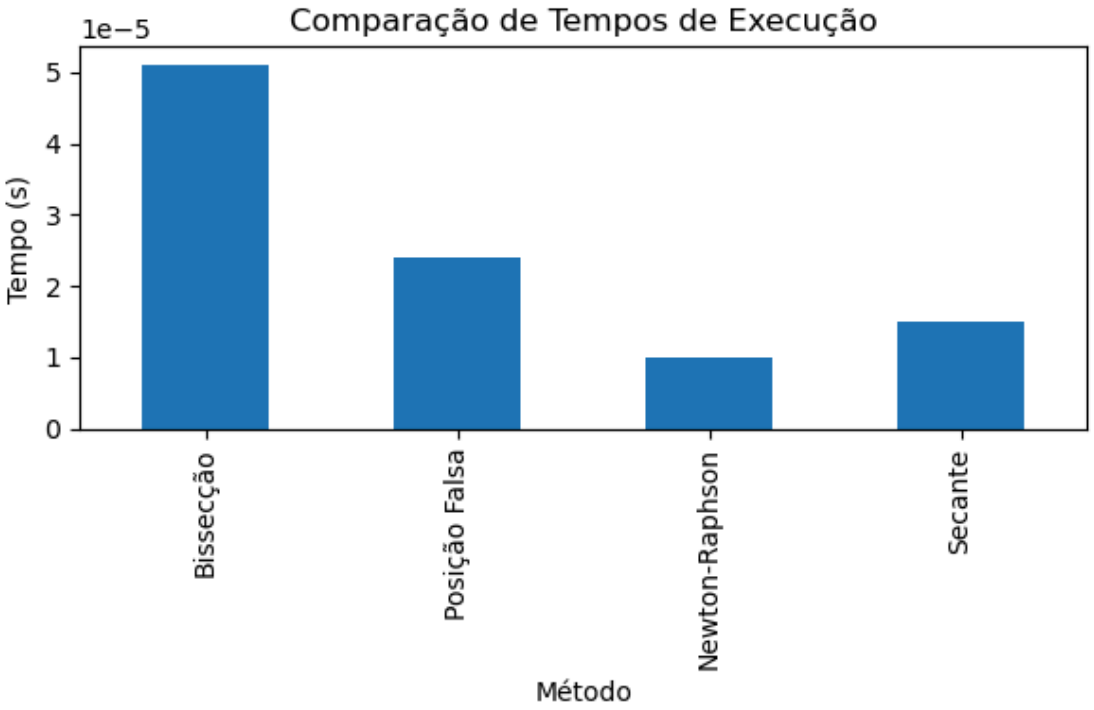
Comparação de erros(Entrada 1):



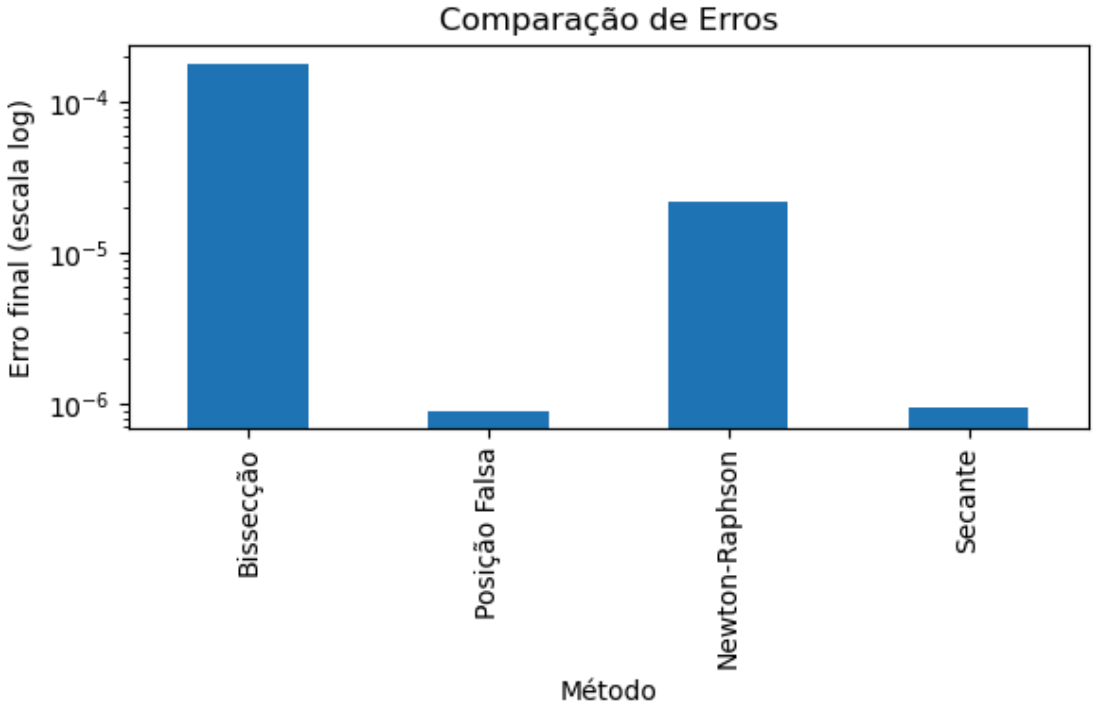
Comparação de iterações(Entrada 1):



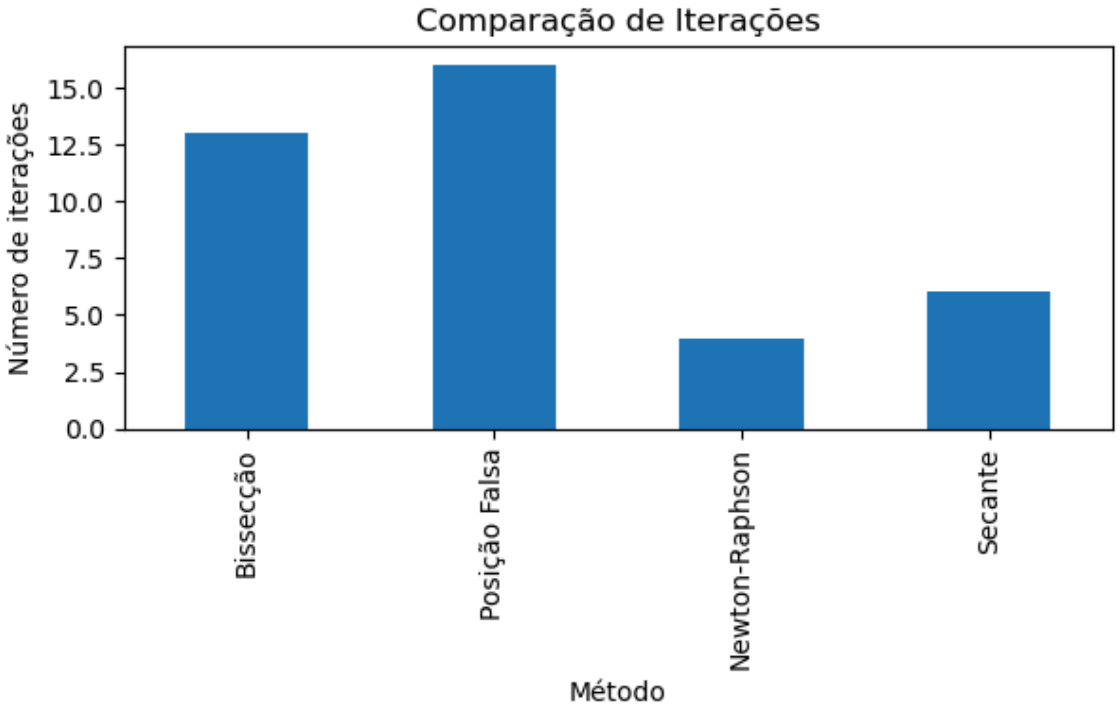
Comparação de tempo(Entrada 1):



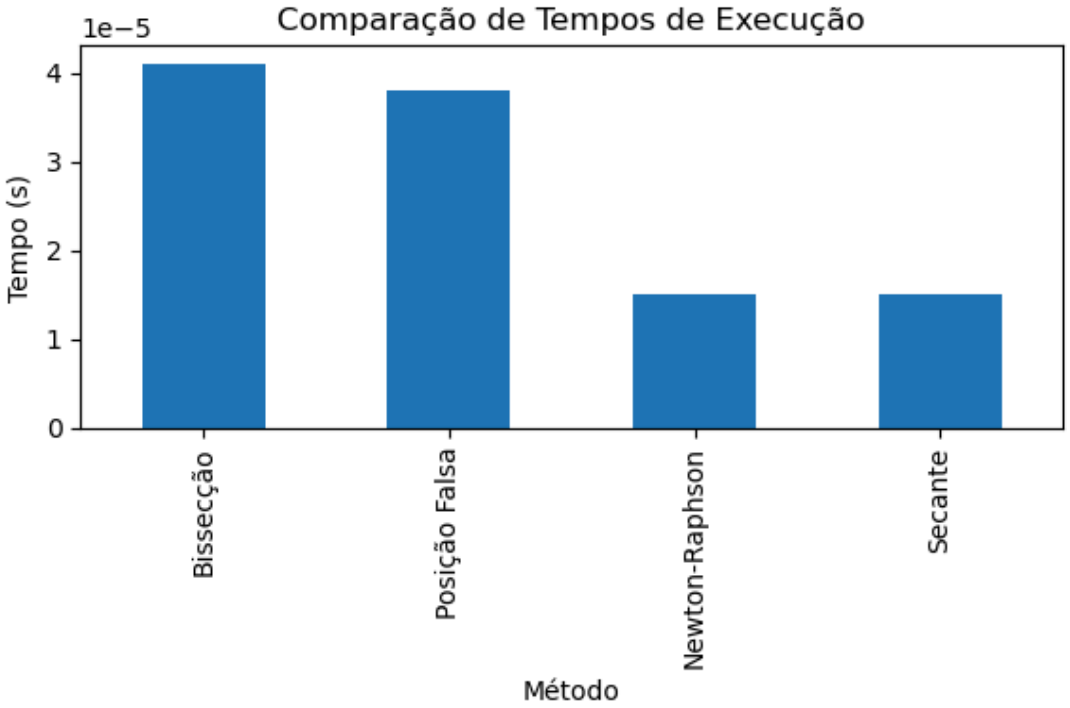
Comparação de erros(Entrada 2):



Comparação de iterações(Entrada 2):



Comparação de tempo(Entrada 2):



Problema 3.6:

3.6 - *A equação:*

$$tg\left(\frac{\theta}{2}\right) = \frac{sen\ \alpha\ cos\ \alpha}{\frac{g\ R}{v^2} - cos^2\ \alpha},$$

permite calcular o ângulo de inclinação, α, em que o lançamento do míssil deve ser feito para atingir um determinado alvo. Na equação acima,

- α - *ângulo de inclinação com a superfície da Terra com a qual é feita o lançamento do míssil* ,
- g - *aceleração da gravidade* $\simeq$ 9.81 m/s^2 ,
- R - *raio da Terra* $\simeq$ 6371000 m ,
- v - *velocidade de lançamento do míssil,* m/s ,
- θ - *ângulo (medido do centro da Terra) entre o ponto de lançamento e o ponto de impacto desejado* ,

Resolva o problema considerando: $\theta = 80^o$ *e* v *tal que* $\frac{v^2}{gR} = 1.25$, *ou seja, aproximadamente* 8.840 m/s.

Entrada 1:

```
(sin(x)*cos(x))/(0.8 - cos(x)**2) - 0.839
1
1.2
1e-6
```

Saída por Bissecção(Entrada 1):

```
1.0238052368164063
7.629394530583866e-07
17
0.000087
```

Saída por Posição Falsa(Entrada 1):

```
1.0238060742346389
3.093779741281111e-07
4
0.000026
```

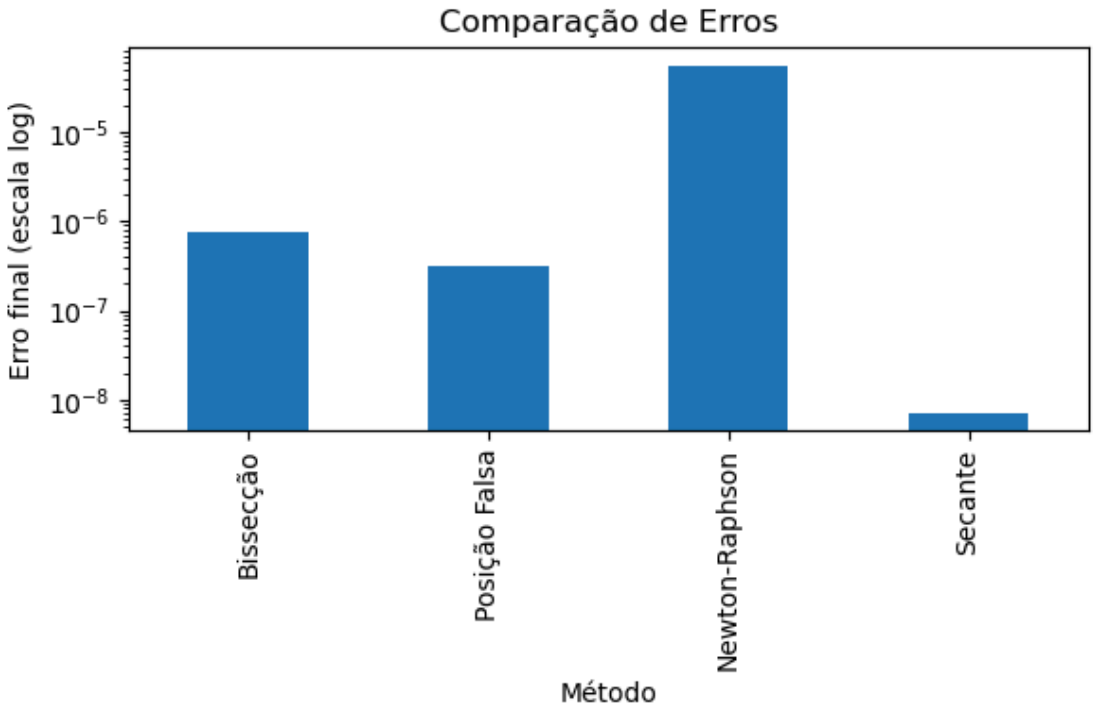
Saída por Newton-Raphson(Entrada 1):

```
1.02380593422818
5.6301916336654045e-05
3
0.000026
```

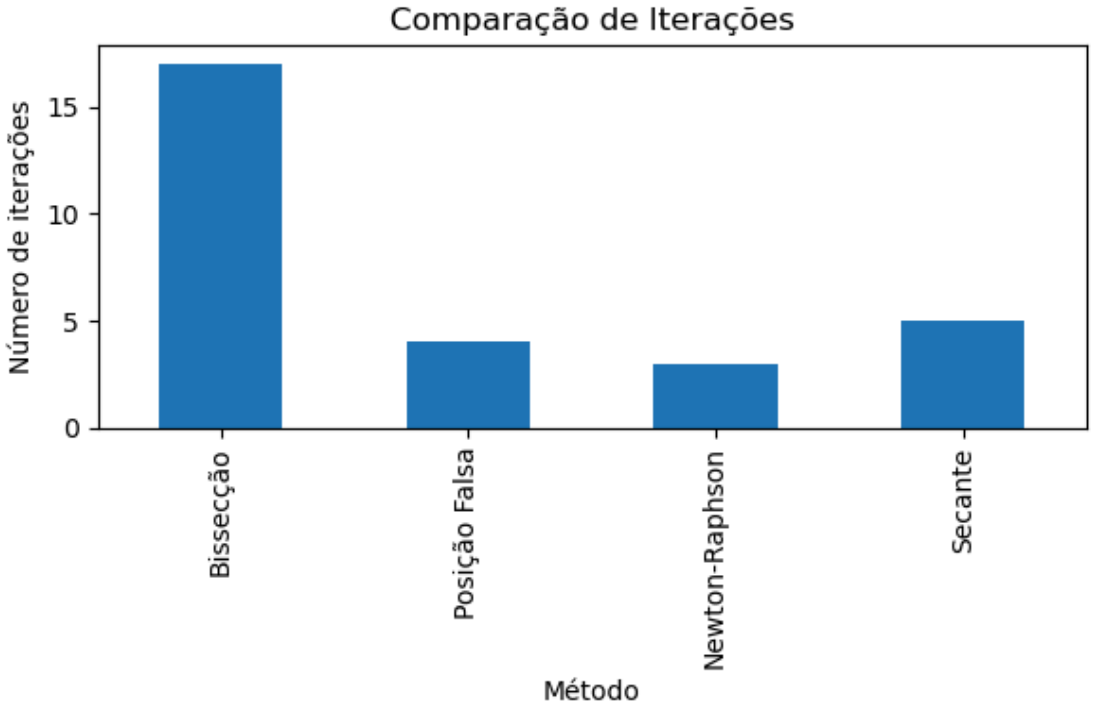
Saída por Secante(Entrada 1):

```
1.0238059382231053
7.190827933101218e-09
5
0.000023
```

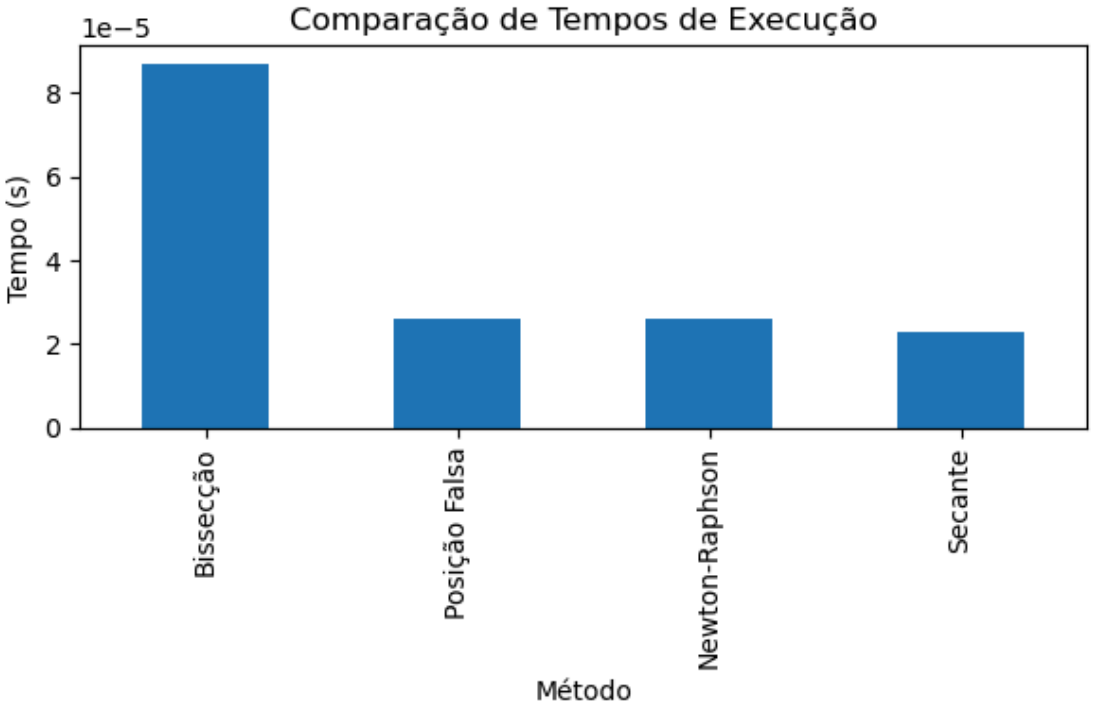
Comparação de erros(Entrada 1):



Comparação de iterações(Entrada 1):



Comparação de tempo(Entrada 1):



Problema 3.8:

3.8 - *Uma loja de eletrodomésticos oferece dois planos de financiamento para um produto cujo preço a vista é R$ 162,00:*

- *Plano A: entrada de R$ 22,00 + 9 prestações iguais de R$ 26,50,*
- *Plano B: entrada de R$ 22,00 + 12 prestações de R$ 21,50.*

Qual dos dois planos apresenta a menor taxa de juros, sendo portanto melhor para o consumidor?

Observação: *Sabe-se que a equação que relaciona os juros* **(J)** *e o prazo* **(P)** *com o valor financiado (***VF** *= preço à vista - entrada) e a prestação mensal* **PM** *é dada por:*

$$\frac{1-(1+J)^{-P}}{J} = \frac{VF}{PM} \ . \tag{3.33}$$

a) *Fazendo* $x = 1 + J$ *e* $k = \frac{VF}{PM}$, *verificar que a equação (3.33) se transforma em:*

$$f(x) = kx^{P+1} - (k+1)x^P + 1 = 0 \ . \tag{3.34}$$

b) *Escrever a equação (3.34) para o problema proposto e encontrar um intervalo contendo a raiz positiva* $\neq 1$.

Entrada 1:

```
5.283*x**10 - 6.283*x**9 + 1
1.1
2.0
1e-6
```

Entrada 2:

```
6.512*x**13 - 7.512*x**12 + 1
1.1
2.0
1e-6
```

Saída por Bissecção(Entrada 1):

```
1.1222490310668947
8.583068847878295e-07
19
0.000014
```

Saída por Posição Falsa(Entrada 1):

```
1.120397976862519
0.01245944650122155
1000
0.000398
```

Saída por Newton-Raphson(Entrada 1):

```
1.1222492519885199
5.061758754365542e-06
9
0.000009
```

Saída por Secante(Entrada 1):

```
1.122249251602054
1.536161713389106e-07
8
0.000007
```

Saída por Bissecção(Entrada 2):

```
1.1093675613403322
8.583068847878295e-07
19
0.000014
```

Saída por Posição Falsa(Entrada 2):

```
1.1031748820589111
0.06603612574036788
1000
0.000414
```

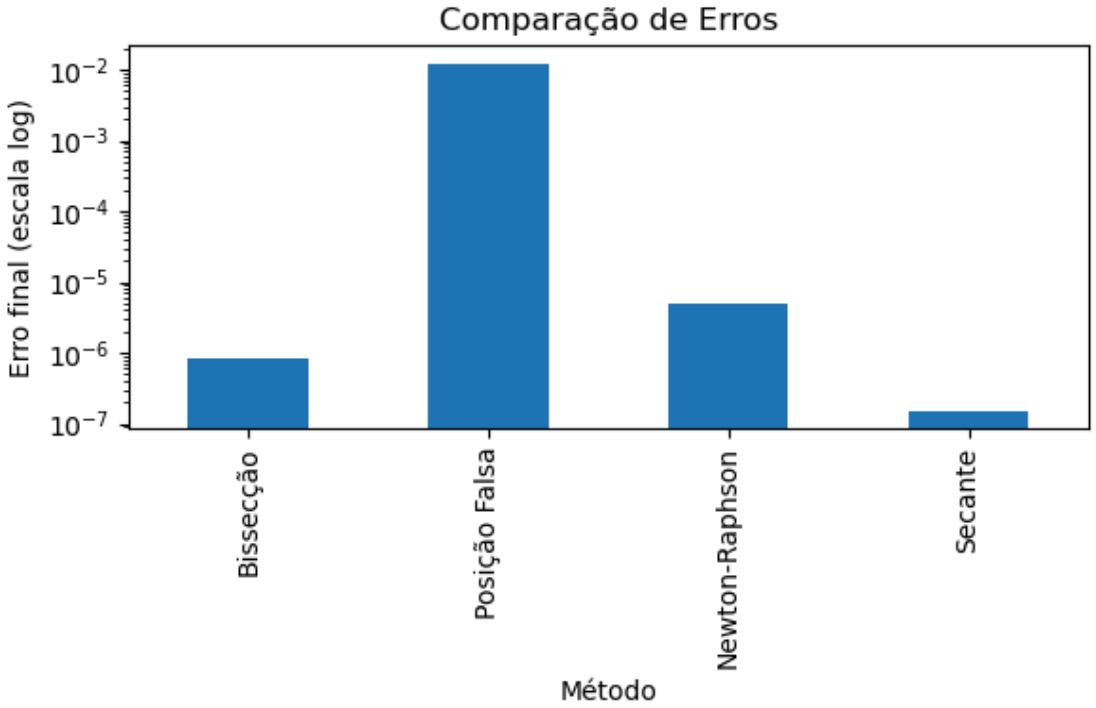
Saída por Newton-Raphson(Entrada 2):

```
1.1093663974170314
2.2507850776287697e-05
10
0.000009
```

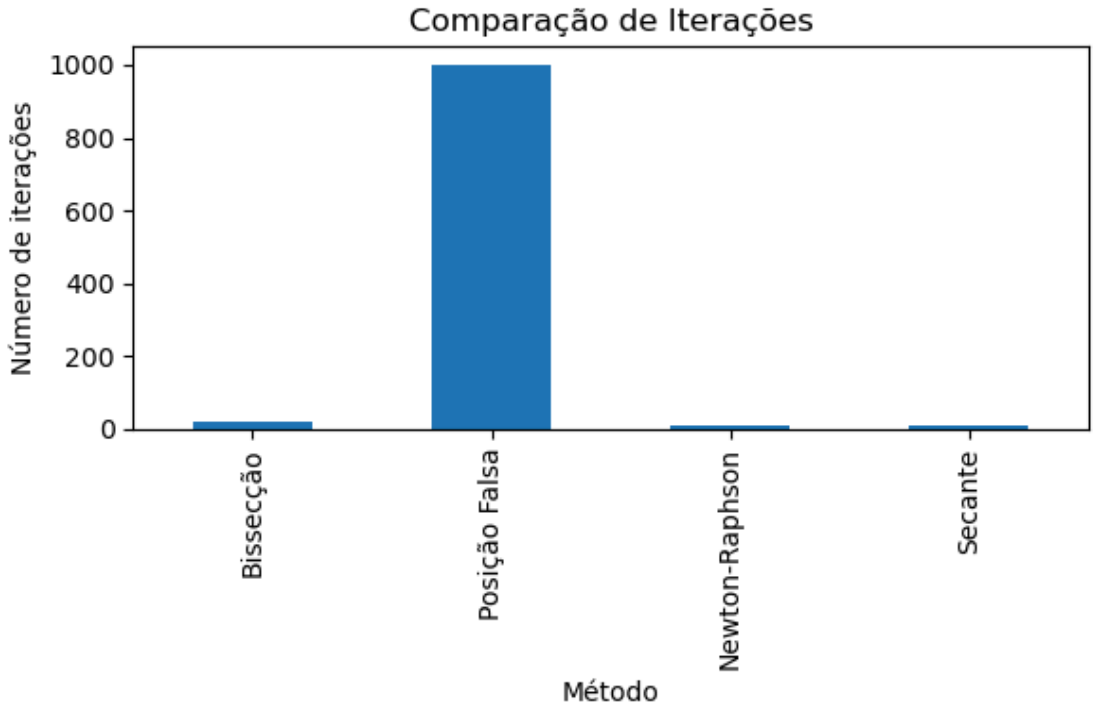
Saída por Secante(Entrada 2):

```
1.1093663892109742
3.9219887115748975e-08
7
0.000006
```

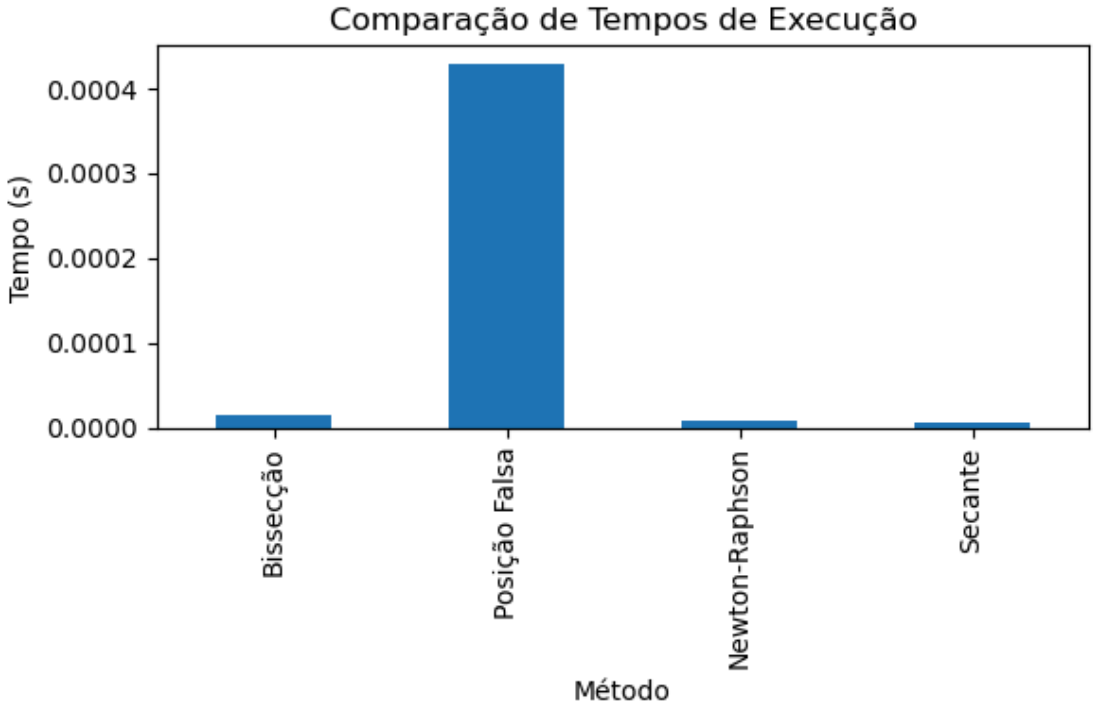
Comparação de erros(Entrada 1):



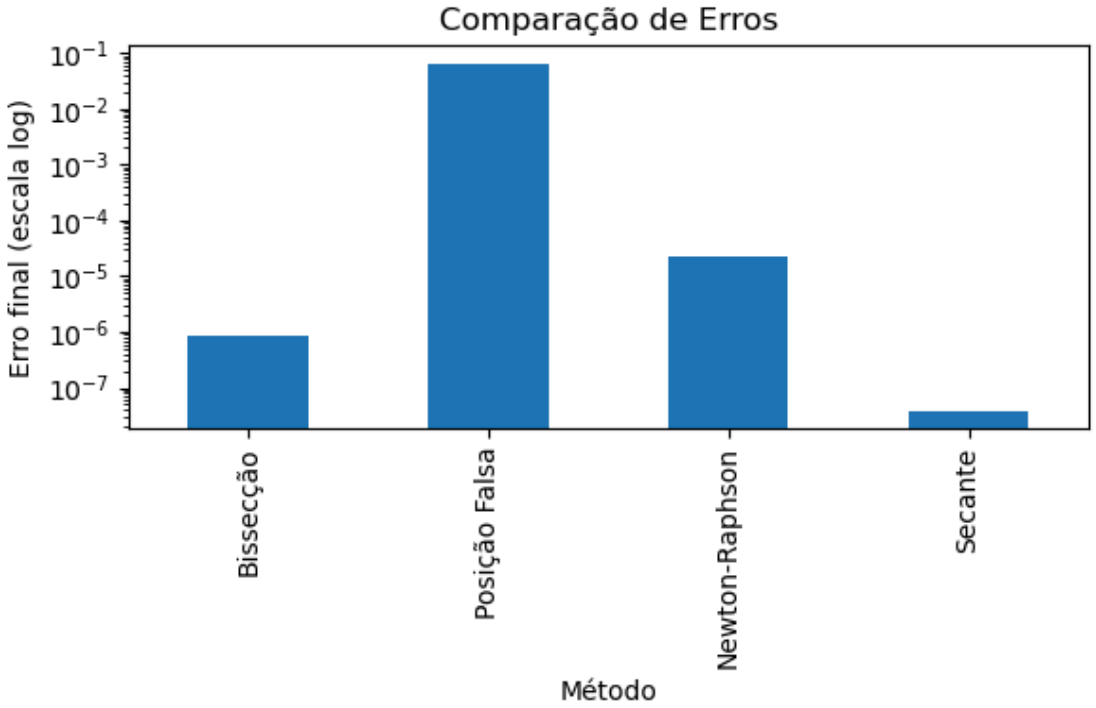
Comparação de iterações(Entrada 1):



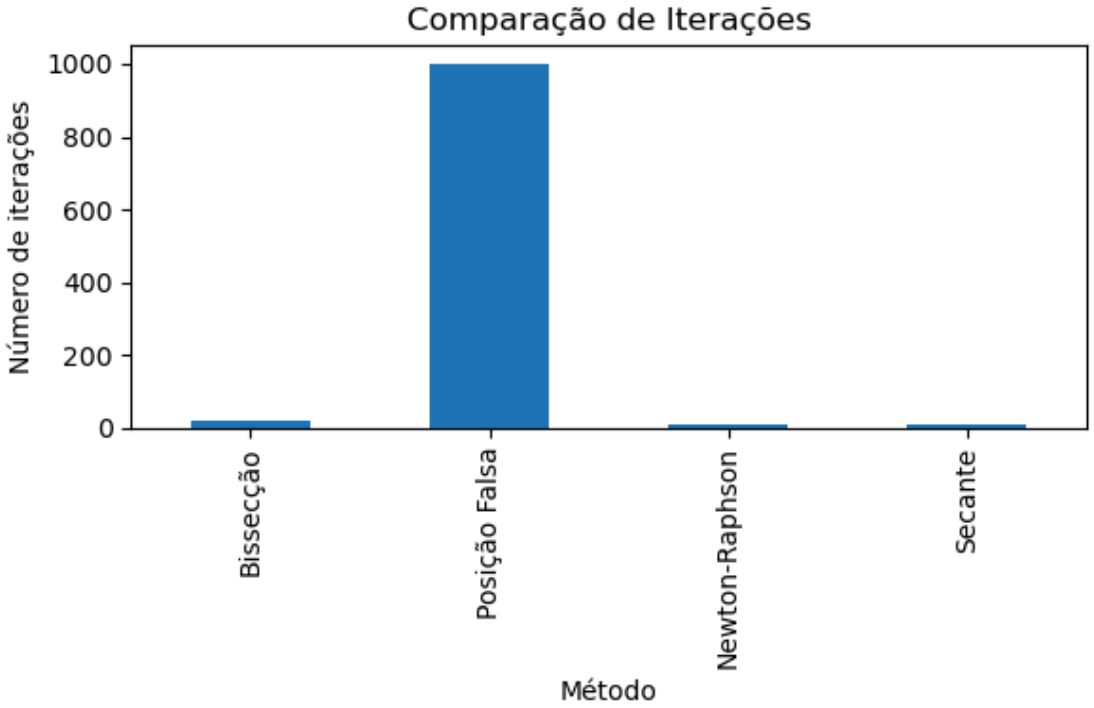
Comparação de tempo(Entrada 1):



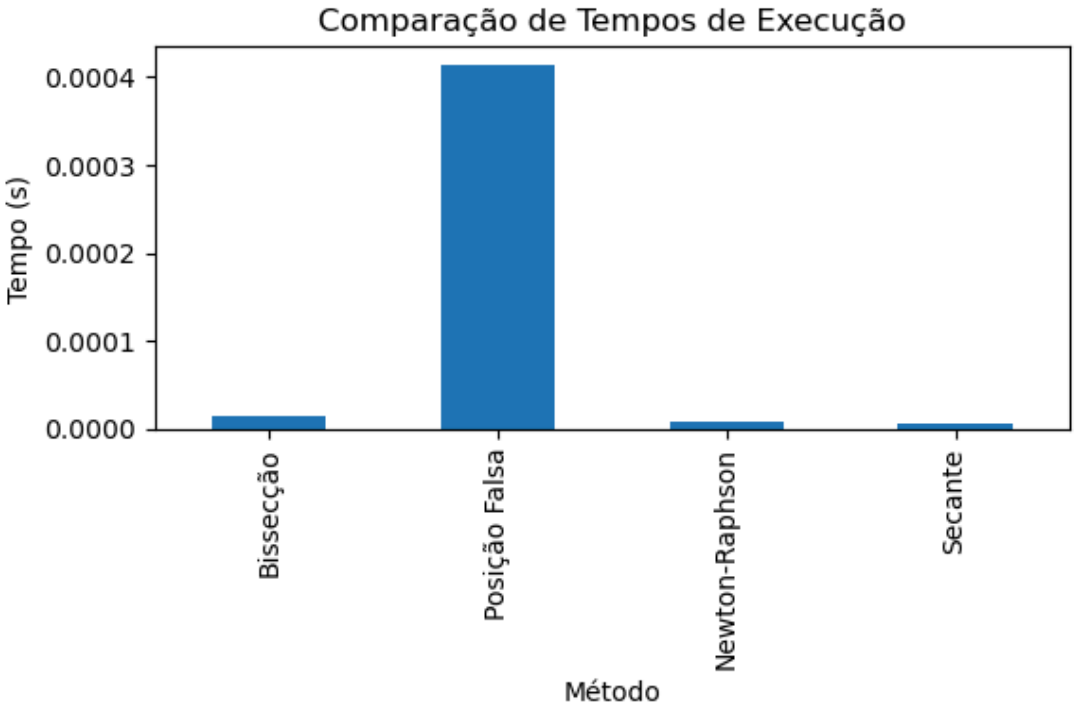
Comparação de erros(Entrada 2):



Comparação de iterações(Entrada 2):



Comparação de tempo(Entrada 2):



Método da Eliminação de Gauss

Estratégia de Implementação:

Implementei a eliminação de Gauss como um método direto sobre a matriz aumentada lida do arquivo: após ler A e b (conforme o formato definido), faço a eliminação para triangularizar A e depois a retro-substituição para obter x. Optei por uma implementação simples e explícita (sem pivoteamento) para deixar o código didático e fácil de acompanhar — isso facilita entender cada passo do processo numa disciplina introdutória. Para robustez incluí verificações de dimensões e conversão para float, e lancei exceções claras quando há problemas (por exemplo pivô zero durante a eliminação ou retro-substituição). O tempo de execução é medido com time.time() e a solução é arredondada para seis casas decimais antes de ser gravada.

Estrutura dos Arquivos de Entrada/Saída

Entrada (**../entrada.txt**): primeira linha contém o inteiro n (dimensão do sistema). As n linhas seguintes contém cada uma n+1 números separados por espaço: os n coeficientes da linha da matriz A seguidos do termo independente correspondente (coluna de b). Linhas em branco são ignoradas para tolerância a formatações simples.

Saída (**saida.txt**): arquivo gerado no diretório de execução contendo, em texto legível, a solução encontrada (lista com componentes arredondados a 6 casas) e o tempo de execução em segundos. Escolhi esse formato porque é direto para leitura humana e fácil de importar em scripts de pós-processamento (relatórios, tabelas ou verificações automáticas).

Observação: para métodos diretos não-iterativos como Eliminação de Gauss e Fatoração LU medi apenas a *solução* e o *tempo de execução*, já que não há iterações nem critérios de convergência a registrar, analisando somente a correção do resultado e o custo (em tempo) da fatoração/eliminação.

Dificuldades enfrentadas

A implementação do método de Eliminação de Gauss apresentou como principal obstáculo o pivoteamento. Sempre que um pivô é zero ou numericamente muito pequeno, o algoritmo pode falhar ou produzir resultados instáveis. Para contornar esse problema, foi adicionada uma verificação que detecta a situação e lança uma mensagem de erro.

Outro ponto importante foi garantir que o arquivo entrada.txt estivesse corretamente formatado, realizando a conversão de strings para valores float de forma segura e emitindo mensagens claras quando ocorriam erros de leitura ou inconsistências no formato.

Além disso, foi observado que, para matrizes mal condicionadas, a solução obtida em float64 pode sofrer perda de precisão.

Método da Fatoração LU

Estratégia de Implementação:

A fatoração LU foi implementada de forma direta sobre a matriz A lida do arquivo: calculei matrizes L (com 1 na diagonal) e U seguindo as fórmulas padrão e, em seguida, resolvi o sistema por substituição para obter x. Optei por uma implementação explícita e legível, facilitando a compreensão didática do processo. Incluí verificações de dimensão e testes para pivôs zero

$$U_{[i,i]} == \ 0.0$$

lançando erros claros nesses casos. O tempo total da fatoração + solução é medido com time.time() e a solução é arredondada para seis casas antes de gravar, para manter o relatório limpo.

Para manter o código simples e robusto tratei erros de leitura (arquivo inexistente, conversão de strings para float, número de linhas incorreto) e gerei exceções descritivas quando a fatoração não é possível sem pivotamento. Essas escolhas equilibram clareza didática e saneamento de entradas, permitindo identificar rapidamente problemas em casos de teste.

Estrutura dos Arquivos de Entrada/Saída

Entrada (**../entrada.txt**) — o formato segue o padrão adotado nos demais métodos e é fácil de editar/manualizar:
1ª linha → inteiro n (dimensão do sistema)
próximas n linhas → cada linha tem n+1 números separados por espaço: os n coeficientes da linha da matriz A seguidos do termo independente correspondente (coluna b). Linhas em branco são ignoradas.

Saída (**saida.txt**) — arquivo gerado no diretório de execução com duas informações principais: a solução do sistema (vetor x arredondado a 6 casas) e o tempo total de execução em segundos.

Observação: para métodos diretos não-iterativos como Eliminação de Gauss e Fatoração LU medi apenas a *solução* e o *tempo de execução*, já que não há iterações nem critérios de convergência a registrar, analisando somente a correção do resultado e o custo (em tempo) da fatoração/eliminação.

Dificuldades enfrentadas

A implementação do método de Fatoração LU apresentou como principal dificuldade a sensibilidade aos pivôs. Sem um pivoteamento correto, o algoritmo falha sempre que um pivô é zero ou numericamente muito pequeno, o que exigiu a detecção desse caso e a geração de mensagens de alerta ao usuário. Uma melhoria futura seria incluir o pivoteamento parcial para tornar o método mais robusto.

Outro desafio foi a estabilidade numérica em matrizes mal condicionadas, já que, mesmo utilizando float64, pode haver perda de precisão nos resultados.

Além disso, foi necessário cuidado extra para garantir que o arquivo entrada.txt estivesse no formato correto e para fornecer mensagens de erro claras em casos de falha de conversão ou formatação, o que contribuiu para uma experiência de uso mais confiável e compreensível.

Problemas Teste (Propostos pelo professor)

Problema 4.1:

4.1 - *Considere o circuito a seguir com resistências e baterias tal como indicado; escolhemos arbitrariamente as correntes e os valores da malha:*

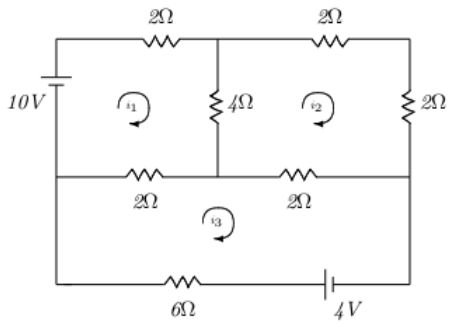


Figura 4.4

Aplicando a Lei de Kirchoff que diz que a soma algébrica da diferenças de potencial em qualquer circuito fechado é zero, obtemos para as correntes i_1, i_2, i_3, o seguinte sistema linear:

$$\begin{cases} 2\,i_1 + 4\,(i_1 - i_2) + 2\,(i_1 - i_3) - 10 = 0 \\ 2\,i_2 - 2\,i_2 + 2\,(i_2 - i_3) + 4\,(i_2 - i_1) = 0 \\ 6\,i_3 + 2\,(i_3 - i_1) + 2\,(i_3 - i_2) - 4 = 0 \end{cases}$$

Deseja-se determinar o valor de $i = (i_1,\ i_2,\ i_3)^t$ que satisfaça o sistema acima.

a) *É possível resolver o sistema pelo método da decomposição LU? Justifique.*

b) *É possível resolver o sistema pelo método de Cholesky? Justifique.*

c) *Resolva o sistema pelo método de Eliminação de Gauss.*

Entrada 1:

```
3
8 -4 -2 10
-4 6 -2 0
-2 -2 10 4
```

Saída por Eliminação de Gauss(Entrada 1):

```
Solução encontrada: [2.758621, 2.310345, 1.413793]
Tempo de execução: 0.000050 segundos
```

Saída por Fatoração LU(Entrada 1):

```
Solução encontrada: [2.758621, 2.310345, 1.413793]
Tempo de execução: 0.000028 segundos
```

Problema 4.3:

4.3 - *Um cachorro está perdido em um labirinto quadrado de corredores (Figura 4.5). Em cada interseção escolhe uma direção ao acaso e segue até a interseção seguinte onde escolhe novamente ao acaso nova direção e assim por diante. Qual a probabilidade do cachorro, estando na interseção i, sair eventualmente pelo lado sul?*

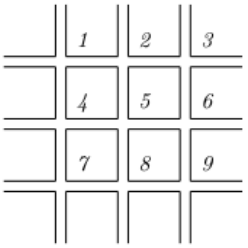


Figura 4.5

Entrada 1:

```
9
4 -1 0 -1 0 0 0 0 0 0
-1 4 -1 0 -1 0 0 0 0 0
0 -1 4 0 0 -1 0 0 0 0
-1 0 0 4 -1 0 -1 0 0 0
0 -1 0 -1 4 -1 0 -1 0 0
0 0 -1 0 -1 4 0 0 -1 0
0 0 0 -1 0 0 4 -1 0 1
0 0 0 0 -1 0 -1 4 -1 1
0 0 0 0 0 -1 0 -1 4 1
```

Saída por Eliminação de Gauss(Entrada 1):

```
Solução encontrada: [0.071429, 0.098214, 0.071429, 0.1875, 0.25, 0.1875, 0.428571, 0.526786, 0.4285
71]
Tempo de execução: 0.000093 segundos
```

Saída por Fatoração LU(Entrada 1):

```
Solução encontrada: [0.071429, 0.098214, 0.071429, 0.1875, 0.25, 0.1875, 0.428571, 0.526786, 0.4285
71]
Tempo de execução: 0.000097 segundos
```

Problema 4.6:

4.6 - *Considere o circuito em escada, mostrado na Figura 4.7, composto de resistores concentrados e fontes de tensão ideais, uma independente e outra controlada.*

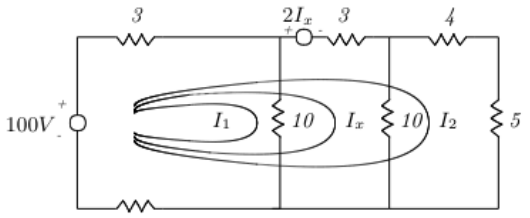


Figura 4.7

O sistema de equações que se obtém ao solucionar o circuito pelo método dos laços é o seguinte:

$$\begin{cases} 4\ (I_1 + I_2 + \ I_x) + 10\ I_1 = 100 \\ 4\ (I_1 + I_2 + I_x) + 3\ (I_2 + I_x) + 10\ I_x = 100 - 2\ I_x \\ 4\ (I_1 + I_2 + I_x) + 3\ (I_2 + I_x) + 9\ I_x = 100 - 2\ I_x \end{cases}$$

a) *Escreva o sistema acima na forma $Ax = b$.*

b) *Determine A^{-1}.*

c) *Usando o resultado obtido em* **b)** *determine o valor de I_x.*

Entrada 1:

```
3
14 4 4 100
4 7 19 100
4 7 18 100
```

Saída por Eliminação de Gauss(Entrada 1):

```
Solução encontrada: [3.658537, 12.195122, -0.0]
Tempo de execução: 0.000049 segundos
```

Saída por Fatoração LU(Entrada 1):

```
Solução encontrada: [3.658537, 12.195122, -0.0]
Tempo de execução: 0.000031 segundos
```

Método de Jacobi

Estratégia de Implementação:

A implementação do método de Jacobi foi feita de forma iterativa, inicializando o vetor x com zeros e atualizando seus valores a cada iteração a partir das fórmulas clássicas. Para simplificar, separei a matriz A em D (diagonal) e R (restante), o que facilita a expressão do método. Implementei um critério de parada baseado na norma infinito da diferença entre x novo e antigo, comparando com a tolerância informada no arquivo (padrão 1e-6). Também limitei o número de iterações a 1000, para evitar loops infinitos caso o método não convergisse.

Tratei erros comuns, como matriz com zeros na diagonal (o que inviabiliza o método), divergência (quando o resultado fica infinito ou NaN) e entradas inválidas no arquivo. Em todos esses casos, o programa gera mensagens claras para facilitar o ajuste pelo usuário. O tempo de execução é registrado com time.time() e, ao final, é escrito em arquivo tanto o vetor solução (arredondado para seis casas) quanto o número de iterações realizadas.

Estrutura dos Arquivos de Entrada/Saída

Entrada (**../entrada.txt**) — formato semelhante aos métodos diretos:
1ª linha → inteiro n (dimensão do sistema)
próximas n linhas → cada linha contém n+1 números separados por espaço (coeficientes da linha da matriz A + termo independente correspondente)
linha seguinte (opcional) → tolerância para critério de parada (padrão 1e-6)
próxima linha (opcional) → número máximo de iterações (padrão 1000)

Saída (**saida.txt**) — contém a solução aproximada do sistema (vetor x arredondado), o número de iterações realizadas e o tempo de execução. Se o método divergir, é gravada uma mensagem informando que não foi possível encontrar solução e quantas iterações foram feitas até detectar a divergência.
O nome fixo saida.txt foi mantido para seguir o padrão dos outros métodos e facilitar a automação de testes.

Dificuldades enfrentadas

A maior dificuldade foi garantir a convergência do método, já que algumas matrizes não satisfazem as condições de convergência de Jacobi (como diagonal dominante). Também foi preciso tratar casos em que surgiam divisões por zero na diagonal e situações de divergência, para que o programa não entrasse em loop infinito. Fora isso, a implementação funcionou bem para os exemplos fornecidos e retornou resultados coerentes.

Método de Gauss - Seidel

Estratégia de Implementação:

Para implementar o método de Gauss–Seidel, iniciei o vetor solução x com zeros e, a cada iteração, atualizei os valores utilizando imediatamente os resultados já calculados dentro da mesma iteração. Isso acelerou a convergência em relação ao método de Jacobi. Como critério de parada, usei a norma infinito do vetor resíduo (A·x - b) comparada com a tolerância (padrão 1e-6). Também incluí um número máximo de iterações (padrão 1000) para evitar loops infinitos caso o método não convergisse.

Tratei casos de erro como presença de zero na diagonal (o que inviabiliza o método), valores inválidos no arquivo de entrada e situações de divergência (quando o resultado se torna infinito ou NaN). Para cada erro, o programa exibe mensagens claras no terminal, ajudando a identificar o problema. O tempo de execução foi medido com time.time() e o resultado foi gravado em um arquivo de saída padronizado (saida.txt), incluindo a solução aproximada, o número de iterações e o tempo total.

Estrutura dos Arquivos de Entrada/Saída

Entrada (**../entrada.txt**) — formato idêntico ao usado no método de Jacobi:
1ª linha → inteiro n (dimensão do sistema)
próximas n linhas → cada linha contém n+1 números separados por espaço (coeficientes da linha da matriz A + termo independente correspondente)
linha seguinte (opcional) → tolerância para critério de parada (padrão 1e-6)
próxima linha (opcional) → número máximo de iterações (padrão 1000)

Saída (**saida.txt**) — registra a solução arredondada do sistema, o número de iterações realizadas e o tempo de execução. Caso o método diverja, é gravada uma mensagem informando que não foi possível encontrar solução.

Dificuldades enfrentadas

A maior dificuldade foi garantir que o método realmente convergisse, já que isso depende da matriz ser diagonalmente dominante ou satisfazer outras condições de convergência. Também foi preciso tomar cuidado com divisões por zero e tratar situações em que o método divergia. Apesar disso, o código funcionou bem para os exemplos propostos e convergiu rapidamente na maioria dos casos.

Condição da Matriz

Estratégia de Implementação:

O algoritmo desenvolvido tem como objetivo calcular a inversa de uma matriz quadrada e determinar seu número de condição utilizando a fatoração LU. O processo começa lendo a matriz de um arquivo de entrada, verificando se ela é quadrada e se os valores podem ser convertidos corretamente para números reais.

Em seguida, a matriz AAA é decomposta em duas matrizes triangulares LLL e UUU. A partir dessa decomposição, sistemas lineares unitários são resolvidos para obter cada coluna da inversa de AAA. O número de condição é calculado como o produto das normas L2 da matriz original e da inversa, fornecendo uma medida da sensibilidade do sistema a pequenas variações nos dados. O tempo total de execução do algoritmo também é registrado, permitindo avaliar sua eficiência.

Para manipulação das matrizes e execução das operações, a implementação utiliza a biblioteca NumPy, que oferece funções otimizadas e confiáveis para cálculos matriciais, garantindo precisão e velocidade. Ao final, todos os resultados — incluindo a inversa, o número de condição e o tempo de execução — são gravados em um arquivo de saída.

Estrutura dos Arquivos de Entrada/Saída

Arquivo de Entrada (**entrada.txt**)

O arquivo de entrada deve conter uma matriz quadrada, com cada linha representando uma linha da matriz e os valores separados por espaços. Todos os valores devem ser numéricos (float).

Arquivo de Saída (**saida.txt**)

Após a execução, o programa gera um arquivo contendo o número de condição da matriz (norma 2), com precisão de seis casas decimais, o tempo de execução do cálculo, em segundos, e a matriz inversa calculada, com valores apresentados com precisão de seis casas decimais.

Observação: diferente dos outros métodos, neste o arquivo de entrada se encontra na mesma pasta do script, uma vez que a entrada para este método se difere dos outros.

Dificuldades enfrentadas

Um dos maiores desafios foi manter a precisão numérica, principalmente ao lidar com matrizes que apresentam elementos muito pequenos ou muito grandes. Para evitar erros, o código verifica se algum pivô é zero ou próximo de zero, lançando exceções quando necessário.

Outra dificuldade foi o cálculo do número de condição, que depende das normas da matriz e de sua inversa. Foi necessário utilizar funções eficientes do NumPy e garantir que operações de grande porte não comprometessem o desempenho ou a exatidão dos resultados.

Problemas Teste (Propostos pelo professor)

Problema 5.1:

5.1 - *Uma maneira de se obter a solução da equação de Laplace:*

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 0 \ ,$$

em uma região retangular consiste em se fazer uma discretização que transforma a equação em um problema aproximado consistindo em uma equação de diferenças cuja solução, em um caso particular, exige a solução do seguinte sistema linear:

$$\begin{pmatrix} 4 & -1 & 0 & -1 & 0 & 0 \\ -1 & 4 & -1 & 0 & -1 & 0 \\ 0 & -1 & 4 & 0 & 0 & -1 \\ -1 & 0 & 0 & 4 & -1 & 0 \\ 0 & -1 & 0 & -1 & 4 & -1 \\ 0 & 0 & -1 & 0 & -1 & 4 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \end{pmatrix} = \begin{pmatrix} 100 \\ 0 \\ 0 \\ 100 \\ 0 \\ 0 \end{pmatrix} .$$

Se desejamos a solução com quatro algarismos significativos corretos, qual dos métodos iterativos que você conhece poderia ser aplicado com garantia de convergência? Resolva o sistema pelo método escolhido.

Entrada 1:

```
6
4 -1 0 -1 0 0 3
-1 4 -1 0 -1 0 0
0 -1 4 -1 0 -1 0
-1 0 -1 4 -1 0 -1
0 -1 0 -1 4 -1 0
0 0 -1 0 -1 4 3
0.0001
100
```

Condição da Matriz(Entrada 1):

```
Número de condição (norma 2): 6.380963e+00
Tempo de execução: 0.000344 segundos

Matriz inversa:
[-3.108271e-01, -1.212388e-01, -1.220697e-01, -5.205842e-02, -5.621302e-02, -3.078182e-02, -1.994209e-02, -1.485585e-02, -8.699484e-03]
[-1.212388e-01, -3.459650e-01, -1.389903e-01, -1.236309e-01, -8.875740e-02, -5.980108e-02, -3.260733e-02, -2.681606e-02, -1.485585e-02]
[-1.220697e-01, -1.389903e-01, -3.492887e-01, -8.460279e-02, -1.360947e-01, -6.332620e-02, -4.716102e-02, -3.260733e-02, -1.994209e-02]
[-5.205842e-02, -1.236309e-01, -8.460279e-02, -3.578623e-01, -1.627219e-01, -1.450963e-01, -6.332620e-02, -5.980108e-02, -3.078182e-02]
[-5.621302e-02, -8.875740e-02, -1.360947e-01, -1.627219e-01, -3.994083e-01, -1.627219e-01, -1.360947e-01, -8.875740e-02, -5.621302e-02]
[-3.078182e-02, -5.980108e-02, -6.332620e-02, -1.450963e-01, -1.627219e-01, -3.578623e-01, -8.460279e-02, -1.236309e-01, -5.205842e-02]
[-1.994209e-02, -3.260733e-02, -4.716102e-02, -6.332620e-02, -1.360947e-01, -8.460279e-02, -3.492887e-01, -1.389903e-01, -1.220697e-01]
[-1.485585e-02, -2.681606e-02, -3.260733e-02, -5.980108e-02, -8.875740e-02, -1.236309e-01, -1.389903e-01, -3.459650e-01, -1.212388e-01]
[-8.699484e-03, -1.485585e-02, -1.994209e-02, -3.078182e-02, -5.621302e-02, -5.205842e-02, -1.220697e-01, -1.212388e-01, -3.108271e-01]
```

Saída por Jacobi(Entrada 1):

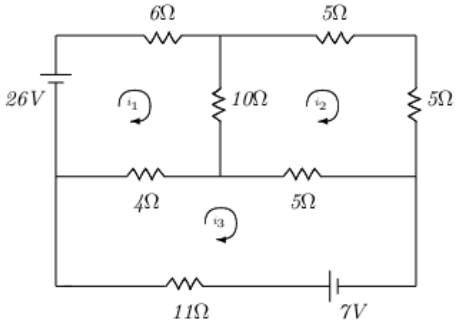
```
Solução encontrada: [0.893847, 0.412765, 0.378661, 0.162765, 0.378661, 0.939311]
Iterações realizadas: 22
Tempo de execução: 0.000222 segundos
```

Saída por Gauss-Seidel(Entrada 1):

```
Solução encontrada: [0.893913, 0.41285, 0.378763, 0.162856, 0.378769, 0.939383]
Iterações realizadas: 14
Tempo de execução: 0.000277 segundos
```

Problema 5.2:

5.2 - *Considere o circuito da figura a seguir, com resistências e baterias tal como indicado; escolhemos arbitrariamente as orientações das correntes.*



Aplicando a lei de Kirchoff, que diz que a soma algébrica das diferenças de potencial em qualquer circuito fechado é zero, achamos para as correntes i_1, i_2, i_3:

$$\begin{cases} 6i_1 + 10(i_1 - i_2) + 4(i_1 - i_3) - 26 = 0 \\ 5i_2 + 5i_2 + 5(i_2 - i_3) + 10(i_2 - i_1) = 0 \\ 11i_3 + 4(i_3 - i_1) + 5(i_3 - i_2) - 7 = 0 \end{cases}$$

a) *É possível aplicar ao sistema acima o método de Gauss - Seidel com convergência assegurada? Justifique.*

b) *Se possível, obtenha a solução com erro relativo* $< 10^{-2}$.

Entrada 1:

```
3
20 -10 -4 1
-10 20 -5 2
-4 -5 20 3
0.0001
100
```

Condição da Matriz(Entrada 1):

```
Número de condição (norma 2): 6.380963e+00
Tempo de execução: 0.000336 segundos

Matriz inversa:
[-3.108271e-01, -1.212388e-01, -1.220697e-01, -5.205842e-02, -5.621302e-02, -3.078182e-02, -1.994209e-02, -1.485585e-02, -8.699484e-03]
[-1.212388e-01, -3.459650e-01, -1.389903e-01, -1.236309e-01, -8.875740e-02, -5.980108e-02, -3.260733e-02, -2.681606e-02, -1.485585e-02]
[-1.220697e-01, -1.389903e-01, -3.492887e-01, -8.460279e-02, -1.360947e-01, -6.332620e-02, -4.716102e-02, -3.260733e-02, -1.994209e-02]
[-5.205842e-02, -1.236309e-01, -8.460279e-02, -3.578623e-01, -1.627219e-01, -1.450963e-01, -6.332620e-02, -5.980108e-02, -3.078182e-02]
[-5.621302e-02, -8.875740e-02, -1.360947e-01, -1.627219e-01, -3.994083e-01, -1.627219e-01, -1.360947e-01, -8.875740e-02, -5.621302e-02]
[-3.078182e-02, -5.980108e-02, -6.332620e-02, -1.450963e-01, -1.627219e-01, -3.578623e-01, -8.460279e-02, -1.236309e-01, -5.205842e-02]
[-1.994209e-02, -3.260733e-02, -4.716102e-02, -6.332620e-02, -1.360947e-01, -8.460279e-02, -3.492887e-01, -1.389903e-01, -1.220697e-01]
[-1.485585e-02, -2.681606e-02, -3.260733e-02, -5.980108e-02, -8.875740e-02, -1.236309e-01, -1.389903e-01, -3.459650e-01, -1.212388e-01]
[-8.699484e-03, -1.485585e-02, -1.994209e-02, -3.078182e-02, -5.621302e-02, -5.205842e-02, -1.220697e-01, -1.212388e-01, -3.108271e-01]
```

Saída por Jacobi(Entrada 1):

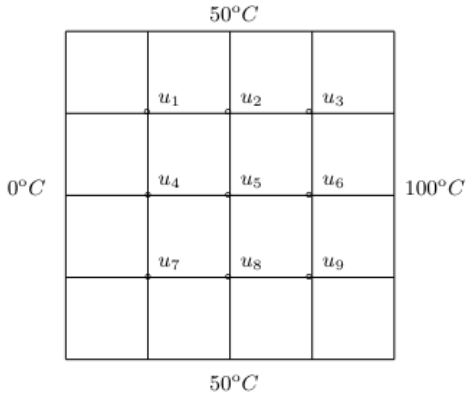
```
Solução encontrada: [0.251948, 0.294413, 0.273958]
Iterações realizadas: 18
Tempo de execução: 0.000196 segundos
```

Saída por Gauss-Seidel(Entrada 1):

```
Solução encontrada: [0.252087, 0.294557, 0.274057]
Iterações realizadas: 14
Tempo de execução: 0.000204 segundos
```

Problema 5.5:

5.5 - *Suponha que uma membrana com dimensões 80 cm × 80 cm tenha cada um dos seus lados mantidos a uma temperatura constante. Usando a teoria de equações diferenciais parciais pode-se formular uma equação que determina o valor da temperatura no interior dessa membrana. Essa equação diferencial pode ser simplificada colocando-se uma malha com 9 pontos sobre essa membrana e calculando-se a temperatura nos pontos da malha, como mostra a figura:*



onde $u_1, u_2, ..., u_9$ são os valores da temperatura em cada ponto da malha.

O sistema de equações resultante é dado por:

$$\begin{pmatrix} -4 & 1 & & 1 & & & & & \\ 1 & -4 & 1 & & 1 & & & & \\ & 1 & -4 & & & 1 & & & \\ 1 & & & -4 & 1 & & 1 & & \\ & 1 & & 1 & -4 & 1 & & 1 & \\ & & 1 & & 1 & -4 & & & 1 \\ & & & & 1 & & -4 & 1 & \\ & & & & 1 & & 1 & -4 & 1 \\ & & & & & 1 & & 1 & -4 \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \\ u_7 \\ u_8 \\ u_9 \end{pmatrix} = \begin{pmatrix} -50 \\ -50 \\ -150 \\ 0 \\ 0 \\ -100 \\ -50 \\ -50 \\ -150 \end{pmatrix}.$$

Encontre a solução do sistema por método numérico à sua escolha, com garantia de convergência.

Entrada 1:

```
9
-4 1 1 0 0 0 0 0 0 1
1 -4 1 1 0 0 0 0 0 2
1 1 -4 0 1 0 0 0 0 3
0 1 0 -4 1 1 0 0 0 4
0 0 1 1 -4 1 1 0 0 5
0 0 0 1 1 -4 0 1 0 6
0 0 0 0 1 0 -4 1 1 7
0 0 0 0 0 1 1 -4 1 8
0 0 0 0 0 0 1 1 -4 9
0.0001
100
```

Condição da Matriz(Entrada 1):

```
Número de condição (norma 2): 6.380963e+00
Tempo de execução: 0.000341 segundos

Matriz inversa:
[-3.108271e-01, -1.212388e-01, -1.220697e-01, -5.205842e-02, -5.621302e-02, -3.078182e-02, -1.994209e-02, -1.485585e-02, -8.699484e-03]
[-1.212388e-01, -3.459650e-01, -1.389903e-01, -1.236309e-01, -8.875740e-02, -5.980108e-02, -3.260733e-02, -2.681606e-02, -1.485585e-02]
[-1.220697e-01, -1.389903e-01, -3.492887e-01, -8.460279e-02, -1.360947e-01, -6.332620e-02, -4.716102e-02, -3.260733e-02, -1.994209e-02]
[-5.205842e-02, -1.236309e-01, -8.460279e-02, -3.578623e-01, -1.627219e-01, -1.450963e-01, -6.332620e-02, -5.980108e-02, -3.078182e-02]
[-5.621302e-02, -8.875740e-02, -1.360947e-01, -1.627219e-01, -3.994083e-01, -1.627219e-01, -1.360947e-01, -8.875740e-02, -5.621302e-02]
[-3.078182e-02, -5.980108e-02, -6.332620e-02, -1.450963e-01, -1.627219e-01, -3.578623e-01, -8.460279e-02, -1.236309e-01, -5.205842e-02]
[-1.994209e-02, -3.260733e-02, -4.716102e-02, -6.332620e-02, -1.360947e-01, -8.460279e-02, -3.492887e-01, -1.389903e-01, -1.220697e-01]
[-1.485585e-02, -2.681606e-02, -3.260733e-02, -5.980108e-02, -8.875740e-02, -1.236309e-01, -1.389903e-01, -3.459650e-01, -1.212388e-01]
[-8.699484e-03, -1.485585e-02, -1.994209e-02, -3.078182e-02, -5.621302e-02, -5.205842e-02, -1.220697e-01, -1.212388e-01, -3.108271e-01]
```

Saída por Jacobi(Entrada 1):

```
Solução encontrada: [-1.930118, -3.103561, -3.617032, -4.867269, -6.434636, -5.931098, -6.323415, -6.42271, -5.436501]
Iterações realizadas: 36
Tempo de execução: 0.000299 segundos
```

Saída por Gauss-Seidel(Entrada 1):

```
Solução encontrada: [-1.930201, -3.103691, -3.617181, -4.867448, -6.434865, -5.931292, -6.323582, -6.42287, -5.436613]
Iterações realizadas: 22
Tempo de execução: 0.000475 segundos
```

Considerações Finais

Este trabalho foi uma oportunidade valiosa de colocar a mão na massa com métodos numéricos, vendo de perto como eles funcionam na prática. Ao lidar com sistemas lineares, cálculo de raízes de funções e manipulação de matrizes, consegui entender não só a teoria, mas também os desafios de implementar esses algoritmos de forma eficiente e precisa.

Mais do que resultados numéricos, a experiência mostrou como pequenas escolhas de implementação podem afetar a convergência e a estabilidade das soluções. Esse aprendizado é uma base sólida para enfrentar problemas mais complexos no futuro, seja em simulações, otimização de processos ou modelagem matemática, tornando claro como a matemática e a programação se conectam para resolver problemas reais.